

Contents

Working-Memory RNN Project	2
How to use this document	3
A minimal mathematical language	3
The implementation question	4
Fast prerequisite route	4
1. Task and datasets	4
1.1 One task trial	4
1.2 The ten-dimensional task cue	5
1.3 Training trials and recorded trials	5
1.4 Human neural datasets	5
2. Model inputs and recurrent architectures	6
2.1 Visual input and output heads	6
2.2 The recurrent cell	6
2.3 Flat network: $S = 0$	7
2.4 Worker-manager network: $S = 1$	7
2.5 What changes when S changes	8
3. The five mechanisms: equations and exact scope	8
3.1 S : hierarchy and sparse local connectivity	8
3.2 M : reflective modulation	9
3.3 P : differentiable Hebbian fast weights	9
3.4 T : topographic activity smoothness	10
3.5 D : soft Dale constraint	10
3.6 Physical scope of each bit	10
4. Training and checkpoint selection	11
4.1 Supervised training	11
4.2 Reinforcement learning	11
4.3 Optimization and curriculum	11
4.4 Behavioral gates	12
4.5 Local-learning study	12
4.6 Main experimental design and behavioral results	12
4.7 Multi-task diet: status and a defensible comparison	13
5. Replay of recorded trials and neural preprocessing	13
5.1 Exact replay and state reset	13
5.2 Session and neuron filtering	14
5.3 Neural time windows	14
5.4 Model time windows	15
6. Analysis map	15
6.1 Data units	15
6.2 Complete analysis map	17
6.3 Pooling and averaging rules	17
7. Each analysis: data unit, output, status, and result	20
Analysis discipline beyond RSA	20
Result snapshot: what remains usable now	20
7.1 Representational analyses	20
7.2 Architectural contrasts and uncertainty	27
7.3 Temporal coding and persistent activity	29
7.4 Encoding models and variance decomposition	30
7.5 State-space geometry	31
7.6 Network properties	32
7.7 Fixed points and local dynamics	33
7.8 Relations among outcome measures	34
7.9 Additional experiments and coded substrates	34
8. Backlog and references	36
8.1 Implementation map and reproducibility	36
8.2 Results backlog derived from the project note	36

Working-Memory RNN Project

Implementation status, results, and analysis backlog

Status date: 12 September 2026

Code revision: 23768b2 audit revision, with uncommitted analysis changes; numerical result tables remain historical saved snapshots.

Evidence: current source code, configuration, saved result tables, replay logs, tests, and the note *There is a credible paper.txt*

This document describes what the repository currently does. It starts with one trial, follows the information through the network, explains how the network is trained and replayed against human recordings, and then defines every analysis in terms of its input rows, distance estimator, aggregation level, saved result, and implementation status.

The network performs a visual Sternberg working-memory task. It sees one to three natural images, waits through a blank delay, sees a probe image, and answers whether the probe was in the remembered set.

Training uses task labels or rewards only. Human neural activity is never a training target. After training, each frozen model is run on the exact image sequences shown in 65 human recording sessions. The model and neural recordings are then compared.

The main experiment changes five properties of a gated recurrent network:

1. *S*: a spatially sparse worker plus a slower manager;
2. *M*: a surprise-dependent bias on a recurrent update gate;
3. *P*: within-trial Hebbian fast weights;
4. *T*: a loss that makes nearby units have similar activity;
5. *D*: a soft penalty that encourages excitatory or inhibitory outgoing recurrent weights.

The principal data flow is

image and task cue $\rightarrow$ frozen ResNet-18 $\rightarrow$ 64-dimensional input $\rightarrow$ recurrent state $\rightarrow$ action and value outputs.

The main neural comparison is

recorded trial $\rightarrow \begin{cases} \text{model epoch-average state,} \\ \text{neural epoch-average firing rates} \end{cases} \rightarrow \text{matched representational comparison.}$

Two trial paths: learn first, replay later

The human recording is a comparison target. It is never an input to the training loss.

TRAINING



FROZEN REPLAY



Figure 1: Training and replay paths

Training uses generated image trials and task feedback; frozen replay uses recorded images and order with the model’s fixed step schedule, while spike-derived firing rates remain a separate comparison target.

How to use this document

This is both the implementation record and a tutorial for a reader who already knows RSA but needs to rebuild the recurrent-network and neural analysis prerequisites. Read Sections 1–5 in order before treating the tables in Section 7 as evidence. Sections 6–7 can then be used as a reference: each analysis says what one row represents, what is fit or compared, what was actually saved, and what remains proposed.

Evidence convention. “Saved result” means a number already present in the repository’s result snapshot. “Implemented” means code exists; it does not imply that it was run on the campaign. “Proposed” means neither a result nor a completed campaign is being claimed. This revision documents the saved snapshot; it does not report a fresh training run, replay, test run, or neural reanalysis.

A minimal mathematical language

A scalar is one number, such as a reward r . A vector is an ordered list, such as a hidden state $h_t \in \mathbb{R}^H$. A matrix maps one vector to another: $Wx + b$ is an **affine** map, where W supplies weighted sums and b shifts them. If $x \in \mathbb{R}^3$, $W \in \mathbb{R}^{2 \times 3}$, and $b \in \mathbb{R}^2$, then $Wx + b \in \mathbb{R}^2$. For example,

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} -2 \\ 9 \end{bmatrix}.$$

Nonlinear functions let the next state depend on the current state in ways that one fixed linear map cannot. Here $\sigma(a) = 1/(1 + e^{-a})$ maps a number to $(0, 1)$, $\tanh(a)$ maps it to $(-1, 1)$, and $\odot$ means elementwise multiplication. An outer product ab^T makes a matrix: its (i, j) entry is $a_i b_j$. Those are all the operations needed for the GRU and fast-weight equations below.

Checkpoint 0. You should be able to say why an affine map changes dimension, why σ can act as a fractional gate, and why a matrix such as $h_t h_{t-1}^T$ can store pairwise pre/post activity. If not, pause here; the rest uses these operations rather than introducing new notation silently.

The implementation question

The scientific question is deliberately narrower than “does an RNN resemble the brain?”: after learning a visual working-memory decision, do different architectural and learning interventions change the geometry and dynamics of the frozen model in ways that match recorded human activity? Behavioral success is necessary, but it is not itself neural alignment. RSA is one comparison of geometries; decoding, encoding, dimensionality, graph metrics, and fixed points answer different questions about the same recorded states.

Fast prerequisite route

Use the shared project learning route for the shortest ordered path; it avoids duplicating a general course inside this project record. For this project, stop after the route’s RNN/GRU, optimization and RL, PCA and demixed-PCA concepts, and local-stability/fixed-point checkpoints. RSA is already assumed. The sibling working-memory dynamics field guide provides the complementary experimental/control context.

1. Task and datasets

1.1 One task trial

The task supplies the reason recurrence is needed. During encoding the network can inspect an image; during maintenance the image input is blank; at probe it must combine the current image with information kept in its state. A feed-forward map of the current input alone cannot solve the blank delay. The recurrent update therefore acts as a learned, finite-dimensional memory.

A trial contains six epochs.

Epoch	Model steps	Input and required behavior
Fixation	3	No image; output action 0
Encoding	$15L$	Present each of L images for 15 steps; output action 0
Maintenance	15	No image; retain the set; output action 0
Probe	10	Present one probe image; answer in-set or out-of-set
Feedback	1	Supply correctness to the reflective and value systems
Intertrial interval	2	No image; output action 0

Here $L \in \{1, 2, 3\}$ is memory load. Trial length is therefore

$$T(L) = 3 + 15L + 15 + 10 + 1 + 2,$$

which gives 46, 61, and 76 steps at loads 1, 2, and 3.

The three actions are:

Action	Meaning
0	fixation or no response
1	yes, the probe is in the remembered set
2	no, the probe is outside the remembered set

The four image categories are faces, animals, objects, and places. On half of ordinary trials, the probe is sampled from the remembered images. On the other half it is a new image. A configured fraction of out-of-set probes are lures: the image is new, but its category matches a remembered category. The target lure fraction is 0.20.

The task generator is `brainalign_wm/tasks/sternberg.py`. The training curriculum is `brainalign_wm/tasks/curriculum.py`.

1.2 The ten-dimensional task cue

At every step the visual feature vector is accompanied by a ten-dimensional context vector c_t .

	Index	Meaning
	0	working-memory task family
	1	auxiliary identity-report trial
	2–4	first, second, or third encoding position; active only during encoding
	5	encoding epoch
	6	maintenance epoch
	7	probe epoch
	8	reserved and always zero
	9	reserved and always zero

Indices 2–4 do not reveal the final load after encoding. The network must carry that information through its recurrent state. Index 8 formerly signaled whether a probe was a lure. That cue perfectly leaked part of the answer and has been removed while preserving the input width for checkpoint compatibility.

Checkpoint 1. On a maintenance step, both the image feature and the position bits are blank. Explain which information must be present in h_{t-1} for the later in/out decision, and why the ten-dimensional cue cannot leak it.

1.3 Training trials and recorded trials

There are two sources of trials.

Training trials are generated from the natural-image bank. They teach the network the task.

Recorded trials come from the human experiments. During post-training replay, the network receives the same remembered images and probe shown in that recorded trial. The model weights are frozen. Neural activity is not fed into the network.

“Replay a trial through the model” therefore means:

1. reset the model state;
2. construct the model’s step sequence from that recorded trial;
3. present the recorded image identities at the correct epochs;
4. run the frozen model forward;
5. save its recurrent state at every step.

1.4 Human neural datasets

Tier A combines DANDI datasets 000469 and 000673.

Quantity	Count
Patients	57
Recording sessions	65
Neurons	2,298
Replayed trials	8,820
Trials per session	108–140
Median trials per session	140

Dataset 000469 contributes 21 sessions: 6 have 108 trials and 15 have 135. Dataset 000673 contributes 44 sessions: one has 128 trials, one has 139, and 42 have 140.

Tier B is DANDI dataset 001187. It is loadable but not connected to the standard analysis. Tier C is DANDI dataset 000574, a verbal working-memory task whose schema requires a separate adapter. Neither Tier B nor Tier C contributes to the results reported here.

The two Tier-A source descriptions are Kyzar et al. (2024) and Daume et al. (2024).

2. Model inputs and recurrent architectures

2.1 Visual input and output heads

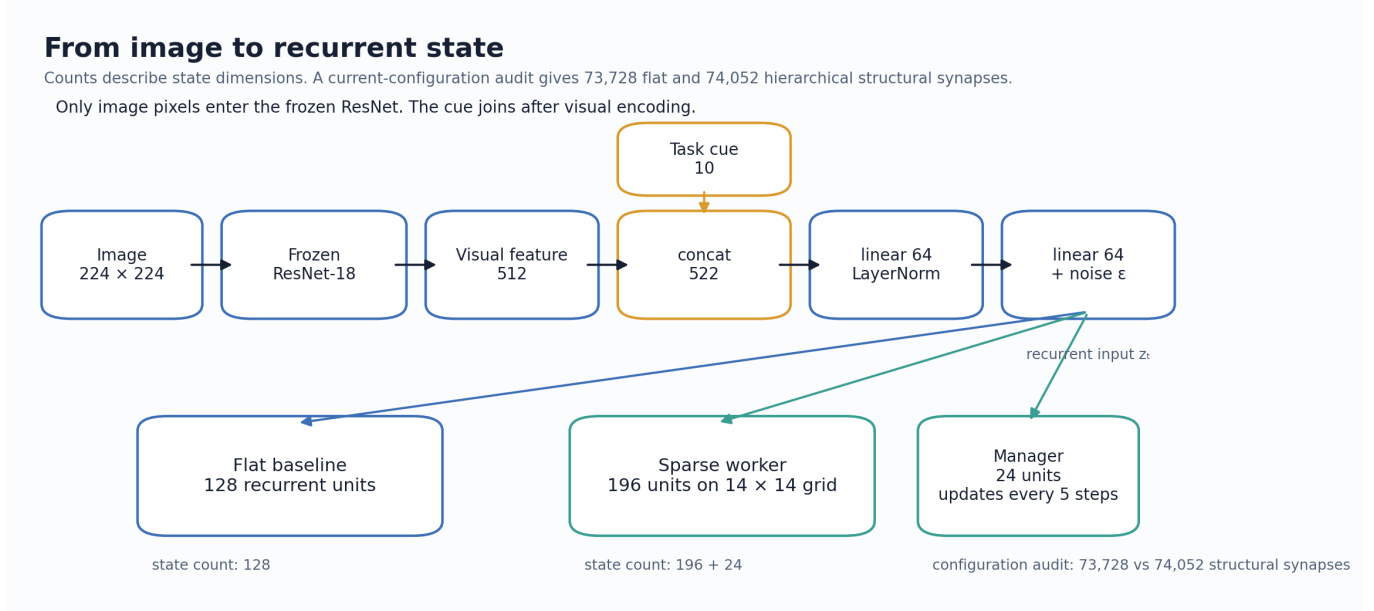


Figure 2: Input and recurrent architectures

Images alone pass through frozen ResNet-18; the 10-D cue joins its 512-D feature before the two 64-D input maps. Current-configuration structural-synapse audit: flat 73,728 vs hierarchical 74,052.

Each image is passed through an ImageNet-pretrained ResNet-18. The classifier is removed, giving a 512-dimensional image feature f_t . The encoder is frozen, so the working-memory training procedure cannot change its weights.

The model concatenates f_t and c_t , then applies **two** learned affine maps with LayerNorm between them. The 522-dimensional concatenated vector first enters a 64-dimensional pre-bottleneck representation a_t , then a second affine map produces the 64-dimensional recurrent input:

$$a_t = \text{LayerNorm}(W_{\text{in}}[f_t; c_t] + b_{\text{in}}), \quad z_t = W_{\text{bottleneck}}a_t + b_{\text{bottleneck}} + \epsilon_t.$$

Here $\epsilon_t \sim \mathcal{N}(0, 0.05^2 I)$ during training and the configured evaluation path. A time step without an image uses the corresponding blank visual input. This second affine layer is in `FrontEnd.forward` in `brainalign_wm/models/front_end.py`; omitting it would describe a different network.

The recurrent readout state h_t^* goes to two shared heads:

$$\pi_t = \text{softmax}(W_{\pi}h_t^* + b_{\pi}), \quad V_t = W_Vh_t^* + b_V.$$

π_t is the distribution over the three actions. V_t predicts trial correctness and serves as the baseline for reinforcement learning and as one input to reflective modulation.

The encoder is `brainalign_wm/encoders/resnet18_encoder.py`. Model assembly and heads are in `brainalign_wm/training/train.py`.

Tiny example: a head with logits $(2, 0, -1)$ assigns the largest softmax probability to action 0, but the other actions retain nonzero probability. That is why a policy can be sampled during RL training, while evaluation can select its largest-probability action deterministically.

2.2 The recurrent cell

The main campaign uses a custom GRU. For input x_t and previous state h_{t-1} ,

$$r_t = \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}),$$

$$u_t = \sigma(W_{iu}x_t + b_{iu} + W_{hu}h_{t-1} + b_{hu} + b_t^{\text{extra}}),$$

$$n_t = \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})),$$

$$h_t = (1 - u_t) \odot h_{t-1} + u_t \odot n_t.$$

A large update gate u_t replaces more of the old state with the candidate state. The extra bias is zero unless reflective modulation is active. The cell exposes u_t for analysis. Its implementation is `brainalign_wm/models/gru_cell.py`.

For one scalar unit, $u_t = 0.9$, $h_{t-1} = 2$, and $n_t = -1$ give $h_t = 0.1(2) + 0.9(-1) = -0.7$: the candidate mostly overwrites the old value. With $u_t = 0.1$, the same candidate gives $h_t = 1.7$, so the unit mostly retains its state. In a real GRU every coordinate can choose a different fraction. This is the mechanism that can preserve information across the blank delay without requiring a literal copy register.

Checkpoint 2. Identify the three roles in the equations: reset gate r_t , update gate u_t , and candidate n_t . Then explain why adding a positive bias to the update-gate *preactivation* tends to make the state change more, not less.

2.3 Flat network: $S = 0$

The flat model has one dense 128-unit GRU:

$$h_t = \text{GRU}(z_t, h_{t-1}), \quad h_t^* = h_t.$$

It updates on every task step. When $T = 1$, these 128 units are assigned to a 16×8 grid for the topographic loss. That grid does not make the recurrent connectivity sparse.

2.4 Worker-manager network: $S = 1$

The hierarchical model has:

Component	Size	Update schedule
Worker	196 units on a 14×14 grid	every step
Manager	24 units	every fifth step
Top-down vector	32 dimensions	after each manager state
Worker summary	49 dimensions from 2×2 mean pooling	every step

The worker receives the task input and the previous manager feedback:

$$h_t^w = \text{GRU}_w([z_t; g_{t-1}], h_{t-1}^w).$$

Worker recurrent connections use a fixed distance-dependent mask. The target density is 0.10, self-connections are excluded, and nearby pairs have higher connection probability.

The 14×14 worker map is mean-pooled in non-overlapping 2×2 blocks:

$$s_t = \text{Pool}_{2 \times 2}(h_t^w) \in \mathbb{R}^{49}.$$

At steps for which $t \bmod 5 = 0$,

$$h_t^m = \text{GRU}_m(s_t, h_{t-1}^m).$$

Between manager ticks,

$$h_t^m = h_{t-1}^m.$$

The manager sends a dense top-down vector back to the worker:

$$g_t = W_g h_t^m + b_g.$$

The action and value heads read both populations:

$$h_t^* = [h_t^w; h_t^m].$$

The implementation is `brainalign_wm/models/hrl.py`.

The words “manager” and “worker” describe information flow and time scale. They do not imply hierarchical reinforcement learning. The manager does not choose goals, options, subpolicies, or separate rewards. Under supervised learning, gradients from the same task loss pass through the readout, manager, feedback projection, and worker. The hierarchy remains meaningful as an architectural constraint: one population updates slowly and compresses worker activity before sending context back.

2.5 What changes when S changes

S is a bundled architectural factor. Turning it on changes all of the following at once:

- one recurrent population becomes two;
- the worker is spatially arranged and sparsely connected;
- worker activity is pooled before reaching the manager;
- the manager sends top-down feedback;
- the manager updates every five steps;
- the readout concatenates worker and manager activity.

An S contrast estimates the effect of this complete package. It does not isolate any one of these ingredients.

3. The five mechanisms: equations and exact scope

3.1 S : hierarchy and sparse local connectivity

$S = 0$ selects the flat 128-unit GRU. $S = 1$ selects the 196-unit worker and 24-unit manager described above. The configured flat core has 73,728 effective structural synapses, and the hierarchical core has 74,052: a 0.44% difference. “Effective structural synapses” counts core input projections, surviving recurrent connections, and the feedback projection; it excludes biases and a plastic mixing coefficient because neither creates a structural synapse. It is not the raw parameter count.

Core / plasticity	Units	Structural synapses	Active core parameters	Raw core parameters
Flat, $P = 0$	128	73,728	74,496	74,496
Hierarchical, $P = 0$	220	74,052	75,404	179,072
Flat, $P = 1$	128	73,728	123,648	123,648
Hierarchical, $P = 1$	220	74,052	86,984	294,320

The raw count includes masked-out weights and, for plastic cells, parameters such as α that modulate existing synapses. It is therefore expected to differ sharply from active or structural counts, especially for sparse plastic workers. The 128-unit flat network is already parameter-matched at the structural-synapse level and already uses the 16×8 layout for T . A 196-unit flat model is an optional wide control, not a mandatory repair or a prerequisite for interpreting the saved campaign.

3.2 M : reflective modulation

Reflective modulation is causal with a one-tick lag. Before the recurrent update at tick t , it builds R_t from quantities logged after tick $t - 1$. Thus the state or action produced at a tick cannot change the gate used at that same tick.

If the **preceding** step was feedback,

$$\delta_t = \text{reward}_{t-1} - V_{t-1}.$$

If the preceding step was not feedback,

$$\delta_t = 1 - p_{t-1}(a_{t-1}),$$

where a_{t-1} is the action selected by the model at that preceding step. The signal is accumulated as

$$R_t = 0.9R_{t-1} + 0.1 \text{softplus}(|\delta_t|).$$

It adds

$$b_t^{\text{extra}} = \beta R_t, \quad \beta = 1,$$

to an update-gate preactivation. Because a larger update gate overwrites more state, larger R_t encourages an update.

The physical target depends on S :

- with $S = 0$, M biases the flat GRU update gate on every step;
- with $S = 1$, M biases the manager update gate only when the manager clock ticks.

The manager clock is the same for $M = 0$ and $M = 1$. This avoids confounding modulation with update frequency.

In particular, a feedback reward is latched when the feedback tick completes and first affects the next tick (normally the first ITI tick). During recorded-trial replay, non-feedback surprise comes from the model’s own chosen-action probability. The reward latched at feedback uses the recorded patient correctness label; it is not an attempt to train the model on neural activity. `ReflectiveGate.surprise`, `ReflectiveGate.step`, and `_gate_bias_step` in `brainalign_wm/mechanisms/reflective_gate.py` and `brainalign_wm/training/train.py` implement this ordering. The manager’s hard five-step clock is unchanged by M : off clock it holds exactly, and on clock M only supplies the additive bias.

3.3 P : differentiable Hebbian fast weights

The slow recurrent matrix W is supplemented by a trial-specific fast trace:

$$W_t^{\text{eff}} = W + \alpha \odot H_{t-1}.$$

After computing h_t , the trace changes according to

$$H_t = \text{clip} \left(0.9H_{t-1} + 0.05h_th_{t-1}^T, -2, 2 \right).$$

A separate trace is stored for each GRU gate block. W and the mixing coefficients α are learned by backpropagation through time. α starts at zero. H_t is not a learned parameter; it is a dynamic state reset at the start of each trial.

This mechanism is “Hebbian” because the fast trace contains products of presynaptic and postsynaptic activity. It is “differentiable plasticity” because gradient descent learns how strongly each trace affects the recurrent computation. It is not a replacement for the network’s training rule.

The physical target again depends on S :

- with $S = 0$, all flat-GRU recurrent blocks receive fast weights;
- with $S = 1$, worker recurrent blocks receive fast weights and the manager remains nonplastic.

Plastic runs use gradient checkpointing to reduce the memory required to backpropagate through the full sequence. The implementation follows the differentiable-plasticity construction in Miconi, Clune, and Stanley (2018).

3.4 T : topographic activity smoothness

For recurrent activity placed on a grid, the loss is the **sum of two separate means**, one for horizontal differences and one for vertical differences:

$$\mathcal{L}_T = \text{mean}[(\Delta_{\text{vertical}}h)^2] + \text{mean}[(\Delta_{\text{horizontal}}h)^2].$$

Equivalently in the implementation, it is `torch.diff(grid_act, dim=1).pow(2).mean() + torch.diff(grid_act, dim=2).pow(2).mean()` at each tick, then averaged over collected ticks. It is therefore not one edge-weighted mean over the union of horizontal and vertical edges; grids with unequal counts in the two directions give each direction equal loss weight.

Its coefficient is 0.01.

- with $S = 0$, the 128 flat units use an imposed 16×8 grid;
- with $S = 1$, the loss acts on the worker’s intrinsic 14×14 grid;
- manager activity is excluded.

T shapes activity during training. It does not create or remove connections. Sparse distance-dependent connectivity belongs to S .

3.5 D : soft Dale constraint

The first 80% of units are assigned an excitatory source identity and the remaining 20% an inhibitory source identity. Columns of a recurrent matrix correspond to source units. The penalty is

$$\mathcal{L}_D = \text{mean ReLU}(-W_{\text{exc}}) + \text{mean ReLU}(W_{\text{inh}}).$$

Its coefficient is 0.01. It is applied to all reset, update, and candidate recurrent blocks. Masked-out worker connections are excluded.

- with $S = 0$, it acts on the flat recurrent matrix;
- with $S = 1$, it acts on worker and manager recurrent matrices.

This is a soft constraint. Training may leave sign violations if the task loss benefits enough to offset the penalty.

3.6 Physical scope of each bit

The five-bit label is compact, but the same bit can act on different populations:

Bit	$S = 0$ target	$S = 1$ target
M	flat update gate	manager update gate
P	flat recurrent weights	worker recurrent weights
T	imposed 16×8 flat grid	worker 14×14 grid
D	flat recurrent weights	worker and manager recurrent weights

This scope is part of the intervention. It must be considered when interpreting add-one and remove-one contrasts.

Checkpoint 3. State the difference between S and T : S creates the worker’s masked local recurrent wiring, whereas T adds an activity-smoothness loss. Then trace a surprising feedback event: it is observed at feedback, updates R on the following tick, and can bias a manager update only if that following tick is also a manager clock tick.

4. Training and checkpoint selection

4.1 Supervised training

Training adjusts parameters to reduce a loss. In supervised WM training the label is known: action 0 outside the decision period and the in/out action at probe. Backpropagation through time (BPTT) is ordinary backpropagation applied after unrolling the recurrent computation across its trial steps. The chain rule assigns credit to a weight for how a small change at an early encoding step could change a later probe loss. It does **not** mean that human firing rates are targets. Here “SUP” always means this working-memory action supervision.

In the SUP condition, the target action is defined at every step. Probe steps have weight 1.0 and all other steps have weight 0.1:

$$\mathcal{L}_{\text{SUP}} = \frac{1}{N} \sum_t w_t \text{CE}(y_t, \pi_t), \quad w_t = \begin{cases} 1, & t \text{ is probe,} \\ 0.1, & \text{otherwise.} \end{cases}$$

The lower weight outside the probe prevents the many fixation targets from dominating training. The complete loss may also contain $\mathcal{L}_T$ and $\mathcal{L}_D$.

4.2 Reinforcement learning

RL has no action label at every decision. It samples actions and uses a reward to raise the probability of actions with positive advantage $r - V_t$. The value head is a learned baseline: it reduces variance in the policy-gradient estimate but is not an oracle. In this Sternberg task the reward is correctness at the probe; the within-trial fixation labels used by SUP are not rewards.

In the RL condition, actions are sampled from π_t during training. A trial receives reward 1 for a correct probe answer and 0 otherwise. REINFORCE is applied throughout the probe:

$$\mathcal{L}_{\text{policy}} = -\frac{1}{N_p} \sum_{t \in \text{probe}} \log \pi_t(a_t) (r - V_t).$$

The value-loss weight is 0.5 and the entropy coefficient is 0.01:

$$\mathcal{L}_{\text{RL}} = \mathcal{L}_{\text{policy}} + 0.5\mathcal{L}_{\text{value}} - 0.01\mathcal{H}(\pi).$$

Evaluation uses the highest-probability action instead of sampling.

Checkpoint 4. Contrast the source of learning information. SUP asks “which action was labeled correct at this tick?”; RL asks “did the sampled action earn reward relative to expectation?” Both still use BPTT through the model dynamics in the main campaign. The separate local-learning study is the exception, not an alternative definition of SUP or RL.

4.3 Optimization and curriculum

Both SUP and RL use Adam with learning rate 10^{-3} and batch size 128. Each batch contains newly generated trials.

Training steps	Loads	Delay	Lures
0–7,999	1	3 steps	none
8,000–15,999	1 and 2	15 steps	none
16,000 onward	1, 2, and 3	15 steps	linearly ramp from 0 to 0.20 over 90,000 steps
after ramp	1, 2, and 3	15 steps	0.20

The main analysis checkpoint is frozen at 24,000 steps. Therefore it occurs only 8,000 steps into the 90,000-step lure ramp. It represents equal training budget, not the final target distribution or a proven representational plateau.

If a run has not met the behavioral gate at 24,000 steps, training may continue to 48,000 steps. Those extension weights are not used for the equal-budget main analysis.

Periodic evaluation occurs every 2,000 training steps with 200 trials per load. Final evaluation uses 500 trials per load.

4.4 Behavioral gates

The inclusion gate is load-1 accuracy of at least 0.83 for three consecutive evaluations. Load-3 accuracy of 0.80 is tracked as an additional milestone. Meeting a gate does not stop equal-budget training.

4.5 Local-learning study

Four separate cells, M00L, M01L, M10L, and M11L, use node perturbation with an e-prop fallback for recurrent learning. They do not use BPTT for recurrent weights. Each has eight seeds.

These models remained near chance, approximately 0.50 at every load. The implementation ran, but the tested local rules did not solve the task. They are excluded from the main five-bit results. The local rules are in `brainalign_wm/mechanisms/local_learning.py`; e-prop follows Bellec et al. (2020).

4.6 Main experimental design and behavioral results

The project does not train every one of the $2^5 = 32$ possible five-bit combinations. It trains 15 selected cells:

- the all-off baseline;
- the all-on model;
- five add-one cells;
- five remove-one cells;
- three two-factor cells: $S + T$, $T + D$, and $S + D$.

There are 120 SUP runs: 15 cells times 8 seeds. All supervised cell means pass the behavioral gate. There are 72 RL runs: 7 flat cells times 8 seeds plus 8 hierarchical cells times 2 seeds. Several hierarchical RL cells remain near chance. The main campaign therefore contains 192 runs.

The saved behavioral table contains load-specific accuracy. “All-load mean” below is the unweighted arithmetic mean of the three load means; it is supplied for orientation and is not a configured training gate.

Signal	Cell	Seeds	Load 1	Load 2	Load 3	All-load mean
SUP	M00000	8	0.9798	0.9435	0.9167	0.9467
SUP	M00001	8	0.9865	0.9505	0.9257	0.9543
SUP	M00010	8	0.9912	0.9667	0.9430	0.9670
SUP	M00011	8	0.9948	0.9698	0.9390	0.9678
SUP	M00100	8	0.9845	0.9535	0.9313	0.9564
SUP	M01000	8	0.9758	0.9405	0.9105	0.9423
SUP	M01111	8	0.9938	0.9765	0.9480	0.9728
SUP	M10000	8	0.9768	0.9410	0.9100	0.9426
SUP	M10001	8	0.9765	0.9407	0.9137	0.9437
SUP	M10010	8	0.9870	0.9570	0.9250	0.9563
SUP	M10111	8	0.9908	0.9655	0.9303	0.9622
SUP	M11011	8	0.9852	0.9560	0.9237	0.9550
SUP	M11101	8	0.9782	0.9440	0.9150	0.9457
SUP	M11110	8	0.9882	0.9595	0.9307	0.9595
SUP	M11111	8	0.9882	0.9597	0.9303	0.9594
RL	M00000	8	0.9745	0.9423	0.8915	0.9361
RL	M00001	8	0.9700	0.9395	0.8708	0.9268
RL	M00010	8	0.9792	0.9540	0.9065	0.9466
RL	M00011	8	0.9210	0.8965	0.8540	0.8905
RL	M00100	8	0.9840	0.9577	0.9340	0.9586
RL	M01000	8	0.7977	0.7635	0.7113	0.7575
RL	M01111	8	0.9237	0.8992	0.8693	0.8974
RL	M10000	2	0.4790	0.5180	0.5260	0.5077

Signal	Cell	Seeds	Load 1	Load 2	Load 3	All-load mean
RL	M10001	2	0.7180	0.7310	0.7130	0.7207
RL	M10010	2	0.7360	0.7140	0.7060	0.7187
RL	M10111	2	0.4780	0.5510	0.5140	0.5143
RL	M11011	2	0.4900	0.5160	0.5200	0.5087
RL	M11101	2	0.7230	0.7380	0.6850	0.7153
RL	M11110	2	0.4880	0.5080	0.5220	0.5060
RL	M11111	2	0.4880	0.5090	0.5230	0.5067

The core behavioral output is available for SUP and RL, for every cell, seed, and load. It is not averaged by patient or session because model evaluation trials are generated independently of the neural dataset.

4.7 Multi-task diet: status and a defensible comparison

The repository also contains a six-task multi-task diet: visual Sternberg, Bandit, Daw two-step, delay match to sample, Go/NoGo, and context decision making. It is a future competence experiment, not part of the saved five-mechanism result snapshot. See `brainalign_wm/tasks/multitask.py` and `scripts/run_multitask_diet.py`.

The chosen future continuation schema keeps the existing **10-entry** input shape, so every WM cue and every first-layer input is preserved exactly. For WM trials, indices 1–7 retain their existing meanings and task bits [0, 8, 9] are 100. For non-WM tasks, indices 1–7 are zero and the three task bits are: bandit 010, two-step 001, delay match 110, Go/NoGo 101, and context decision 011. This coding is a planned schema; no function has been changed to implement it yet.

The existing implemented 13-dimensional path remains a versioned alternative for readers and experiments, not a forced shape change or a reason to retrain the saved WM campaign. Multi-task competence still needs additional learning because the new observations, contingencies, and rewards were never learned by the WM-only checkpoints. Bandit and Daw two-step are reward-only and therefore require RL terms; their absence of a class label must not be papered over by calling all diet training “supervised.”

The primary proposed comparison is paired continuation from each existing checkpoint: one branch continues with the mixed diet and one WM-only control continues from the identical checkpoint. Match **total optimizer updates** as the primary exposure budget, record the number of WM trials/updates actually encountered in each branch, and report WM behavior plus each new task’s competence. The mixed branch will receive fewer WM updates by design; this primary comparison does not also claim equal WM exposure.

Before that study, audit and repair the diet runner, then run representative paired checkpoints rather than a broad grid. Its current NeuroGym path calls the recurrent step with `gate_bias=None`, does not route topographic-loss arguments, and uses immediate reward terms that may not provide delayed credit assignment for the two-step task. In addition, its global step schedule changes the amount of WM exposure during interleaving. These are implementation gaps in a proposed pipeline, not negative experimental results. No diet result is claimed here.

5. Replay of recorded trials and neural preprocessing

5.1 Exact replay and state reset

For every recorded trial, replay:

1. resets the hidden state;
2. resets any Hebbian trace;
3. feeds the recorded remembered images and probe;
4. lets the frozen model choose its own actions;
5. uses recorded patient correctness at feedback for M ;
6. logs full, worker, manager, gate, policy, and value states where available.

Trials do not carry hidden or plastic state into the next trial. The implementation is `brainalign_wm/training/generate_activity_logs.py`.

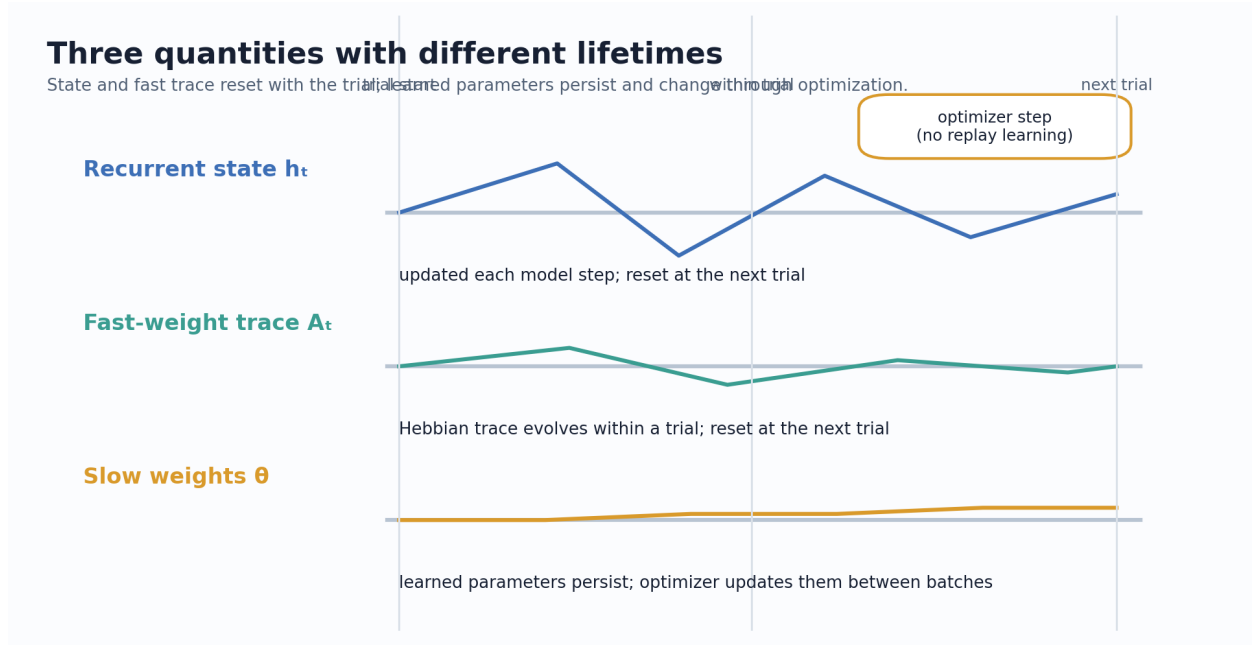


Figure 3: State, weights, and trace timescales

State and fast weights reset at trial boundaries, whereas learned parameters persist and optimizer updates occur between batches.

5.2 Session and neuron filtering

Sessions below 0.55 behavioral accuracy are removed. Neurons below 0.1 Hz mean firing rate are removed. These filters produce the Tier-A counts reported in Section 1.4. The adapter is `brainalign_wm/neural/adapters/dandi_nwb.py`.

5.3 Neural time windows

Spikes are counted in 50 ms bins after the recorded epoch onset.

Epoch	Window	Number of bins
Fixation	0.30 s	6
Maintenance	1.50 s	30
Probe	0.50 s	10

For neuron u , trial i , and bin b ,

$$x_{uib} = \frac{N_{uib}}{0.05 \text{ s}}.$$

The main analyses average within the selected epoch:

$$\bar{x}_{ui} = \frac{1}{B} \sum_{b=1}^B x_{uib}.$$

Each trial therefore contributes one neural vector for maintenance and one for probe.

Timestamp audit of the recorded task The saved alignment sessions have now been audited from their timestamps by docs/audit_tutorial_timings.py; its machine-readable summary is docs/tutorial_timing_audit.json. The two source datasets do not have the same per-item encoding duration.

Dataset	Sessions / trials	Per-item encoding, median (mean; range), s	Valid maintenance delay, median (mean; range), s	Response-latency median, s
000469	21 / 2,673	1.016655 (1.236448; 1.002030–2.510285)	2.684176 (2.680362; 2.534278–2.836186)	1.123749
000673	44 / 6,147	2.015781 (2.012331; 2.003531–2.036030)	2.690061 (2.691842; 2.530500–3.203467)	1.230687

The response values are reaction latencies, not probe-stimulus durations. The audit found no **valid** maintenance delay shorter than 2.530500 s. It also found one nonpositive, malformed 000469 delay timestamp; this trial requires explicit QC rather than use as evidence about timing. The earlier claim that all recorded item presentations were about one second was wrong. Nor does the audit support the earlier rationale that a 1.5-s window was necessary to avoid short valid delays. The current 1.5-s window remains the saved-analysis definition, but a neutral sensitivity analysis over more of the valid delay can use frozen checkpoints and requires no retraining.

The observed ratio of median delay to median item duration differs by source (about 2.64 for 000469 and 1.33 for 000673), whereas the model assigns equal 15-step encoding-item and maintenance segments. These ratios describe separately summarized observations; they are not a prescription for a new common model step count, and the median of per-trial ratios has not been computed.

5.4 Model time windows

The model produces one state per task step. Main maintenance analyses average its 15 maintenance states. Main probe analyses average its 10 probe states.

The 15 model maintenance steps are not paired with the 30 neural maintenance bins. Both sides are independently averaged into one vector per trial. Likewise, the model and neural probe windows both happen to contain 10 samples, but the main analysis does not treat them as synchronized time points.

6. Analysis map

6.1 Data units

An analysis starts by choosing a row unit; this prevents a common error of calling every result “an RSA over sessions.” A trial vector is activity from one trial after averaging a chosen epoch. A condition vector is an estimate made by pooling repeated trials with the same label. A session is a separate recording context, not a feature dimension. A run is one trained seed and cell. These levels answer different sampling questions and must not be interchanged after the calculation.

Before reading an analysis, ask four questions:

1. What is one observation or condition pattern?
2. Which labels are available before the estimate is fit?
3. What split protects the reported number from fitting noise or selection?
4. What population is averaged or resampled for uncertainty?

For example, a probe pseudopopulation has a 12-condition RDM, not 65 session-level RDM rows: each neuron contributes its own within-session condition means, then those features are concatenated. Conversely, maintenance RSA retains one independently estimated condition RDM per session and only then averages its correlations.

Aggregation choices define the comparison

This diagram names the array shapes and two valid routes; it intentionally shows no outcome.

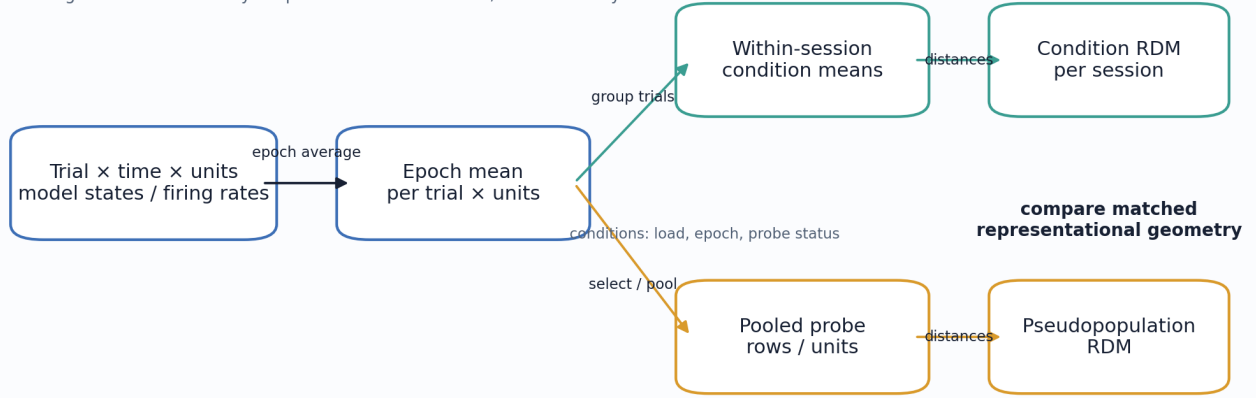


Figure 4: RDM aggregation routes

Epoch means yield either session-level condition RDMs or pooled-probe pseudopopulation RDMs; aggregation must be declared before comparison.

The data are nested:

patient $\supset$ session $\supset$ trial $\supset$ time bin.

This report names an RSA by the rows of its RDM:

- a trial-level RDM has one row per trial;
- a condition-level RDM has one row per condition;
- a session-level RDM has one row per recording session.

The main maintenance analysis produces one RSA value for each session, but its RDM rows are remembered-content conditions. It is therefore condition-level RSA computed separately by session. The project has no 65×65 session RDM.

Analysis	One RDM row is	RDM size	Role of sessions
Main maintenance RSA	remembered-content condition within one load	$C_s \times C_s$	one pair of RDMs and one correlation per session
Main probe RSA	(load, membership, correct) condition	12×12	sessions contribute neurons and within-session condition estimates to one pseudopopulation
Binary maintenance baseline	one trial	$n_s \times n_s$	one pair of RDMs per session
Session-level RSA	one session	65×65	not implemented

Suppose session s has $n_s = 140$ trials and $C_s = 25$ usable remembered-set conditions. All 140 trials may contribute to cross-validated condition estimates, but the final RDM is 25×25 . Pooling repeated trials within each condition does not make the RDM trial-level.

6.2 Complete analysis map

Analysis group	Inputs	Main output	Execution status
Maintenance RSA	per-session trial vectors grouped by remembered content	one signed RSA per session, then a run mean	complete for 192 runs
Probe RSA	12 task conditions and a cross-session neural pseudopopulation	one signed RSA per run and region	complete for 192 runs
RSA controls	encoder, binary labels, untrained net, shuffled reflection, task RDMs	baseline or intervention RSA	mixed; see Section 7.1
Architectural inference	run RSA and accuracy with matched seeds and patient clusters	contrast, interval, FDR result	equal-budget complete
Temporal coding	time-resolved model and neural activity	decoding matrix and stability statistics	old outputs; corrected rerun pending
Persistence	fixation and maintenance activity by unit	standardized cross-validated effect	corrected code; rerun pending
Encoding models	matched model and neural trial vectors	cross-validated R^2	complete
Condition-marginalized PCA	condition-mean activity	marginal variance fractions	complete
State-space geometry	model maintenance trajectories	speed, participation ratio, other geometry	basic measures complete
Network properties	trained recurrent weights and model activity	entropy, modularity, small-worldness, assortativity, selectivity	basic measures complete
Fixed points	recurrent transition and observed states	fixed points and Jacobian spectra	complete
Cross-outcome analysis	run-level behavior, alignment, dynamics, and graph measures	correlations and regressions	input table complete; final output incomplete
Capacity and distillation	separately trained small networks	behavior, dimensionality, teacher match	descriptive results complete
Alternative substrates and causal tests	rate RNNs or interventions	behavior and neural alignment	code, smoke tests, or proposal only

6.3 Pooling and averaging rules

The repository uses “pool” and “mean” for different operations. They are not interchangeable.

1. **Pool trials before estimation.** Repeated trials are used to estimate a condition pattern. The RDM row remains a condition.
2. **Pool neurons into a pseudopopulation.** Probe condition patterns concatenate neurons recorded in different sessions after computing each neuron’s condition mean inside its own session.
3. **Mean over sessions.** Maintenance RSA gives each usable session one r_s , then takes an equal-weight session mean. A patient with several sessions contributes several terms to that point estimate.
4. **Resample patients for uncertainty.** Maintenance confidence intervals sample patients and carry all of their sessions together. This accounts for dependence, but it is not a table of equal-weight patient means.
5. **Mean over seeds or runs.** A cell mean averages its trained seeds. An “all-run” mean averages runs and therefore weights model cells according to their seed counts.
6. **Mean over training signals.** Because the campaign has 120 SUP and 72 RL runs, an all-run mean weights SUP more heavily. It is descriptive. SUP and RL means are reported separately below.

For anatomical pooling, “pooled” means that all eligible Tier-A neurons are used in one regional selection. It is not the arithmetic mean of the ROI-specific RSA values. MFC combines dACC, preSMA, and vmPFC neurons. MTL

combines hippocampus and amygdala neurons. A named ROI uses only sessions containing eligible neurons in that ROI, so ROI means have different session counts.

The saved aggregation variants are:

Analysis	SUP and RL	Cell and seed	Load variation	ROI variation	Worker/manager	Session output	Patient output	Overall mean
Behavior	both	all cells and seeds	loads 1, 2, 3	not applicable	not applicable	not applicable	not applicable	all-load mean can be computed
Maintenance RSA	both	all cells and seeds	distances stratified by load; no per-load RSA saved	pooled, MFC, MTL, dACC, preSMA, vmPFC, amygdala, hippocampus	all units; worker and manager for hierarchical models, region-specific only	yes, 65 when usable	patient label and clustered interval; no patient-mean table	session-weighted run, cell, signal, and all-run means
Probe RSA	both	all cells and seeds	load is part of 12 conditions; no per-load RSA saved	same eight ROI definitions	all units only	no per-session RSA	no patient-level RSA	one pseudopopulation result per run and ROI; cell, signal, and all-run means
Encoder and binary baselines	neither; shared data controls	no model cell or seed	maintenance condition definition	same eight ROI definitions	not applicable	yes where ROI has neurons	patient label saved; no patient-mean table	session-row mean
Untrained control	one untrained M11110 initialization and one SUP trained match	no RL trained match	no per-load RSA	pooled only in saved summary	all units	maintenance uses 65 sessions	no patient summary	single-control value
Reflection shuffle	both	every $M = 1$ run present: 48 SUP and 24 RL	no load split	pooled maintenance only	target follows M : flat or manager	65 sessions enter each run effect	no saved patient-specific effect	signal means
Task controls, sensitivity, epoch-by-method	code only	intended per run	method-dependent	intended pooled and/or region variants	all units	method-dependent	clustered intervals implemented where required	no saved results

Analysis	SUP and RL	Cell and seed	Load variation	ROI variation	Worker/manager	Session output	Patient output	Overall mean
Architectural contrasts	both, separately	paired seeds	accuracy matching uses load 3	pooled neural target	all-unit RSA	maintenance session rows enter bootstrap	patient-clustered uncertainty	all, discovery, and confirmation folds where applicable model and neural means per run and signal row means by cell or signal are derivable
Temporal decoding and persistence	both	all cells and seeds	load labels enter decoder or preference	pooled neural data	no worker/manager split	20 sessions per run in old output	no patient summary	run, cell, signal, and all-run means
Encoding and condition-marginalized PCA	both	all cells and seeds	load is predictor or marginalization, not a per-load result	pooled only	no subpopulation split	one row per run-session	patient label saved; no equal-patient summary	run, cell, signal, and all-run means
Geometry	both	all cells and seeds	no saved load split	model only	full readout state; no separate subpopulation result	not applicable	not applicable	run, cell, signal, and all-run means
Network properties	both	all cells and seeds	no load split	model only	flat, worker, and manager for weight metrics; one whole-model selectivity value	not applicable	not applicable	run, population, cell, signal, and all-run means
Fixed points	both	all cells and seeds	maintenance states seed search; no load split	model only	joint system plus worker/manager Jacobian summaries	not applicable	not applicable	run, cell, signal, and all-run means
Capacity, distillation, Yang reference	separate studies	one or few runs per configuration	task-specific	model only	no common split	not applicable	not applicable	configuration means only

No current analysis gives one equal-weight mean per patient and then averages those patient means. Patient is used as the resampling cluster for maintenance contrasts. No current RSA uses a session-by-session 65×65 RDM.

An equal-session mean and an equal-patient mean are different estimands, not one correct calculation and one error: the former weights recording sessions, whereas the latter weights patients. Likewise, averaging seed-level RDMs before an RSA comparison is a valid ensemble-representation estimand, while averaging seed-level RSA scores estimates a

typical trained run. A report must name which target it uses; this saved snapshot generally uses the session-weighted or run-weighted quantities stated above.

7. Each analysis: data unit, output, status, and result

Analysis discipline beyond RSA

The remaining analyses answer distinct project-specific questions: decoding asks whether a variable can be read out, encoding asks whether one population predicts another, PCA and condition-marginalized PCA organize variance, dynamics tracks state change, graph metrics summarize trained weights, and fixed points study a locally frozen transition. A positive result in one does not imply a positive result in another.

Cross-validation is the repeated discipline behind several of these tools. Fit on one part of the trials and score on withheld trials. For a tiny example, selecting the “best load” of a unit using all trials and reporting its same-data effect will exaggerate persistence; selecting on one half and scoring on the other avoids that circularity. Likewise, ridge regression chooses its penalty inside a training fold and scores predictions on an outer held-out fold.

Result snapshot: what remains usable now

The saved epoch-wise GRU RSA results in Section 7.1 remain the observed campaign snapshot and are retained as such. A broader neural reanalysis of the full recorded delay is a **post-training** task: it needs frozen-checkpoint replay/log selection and neural preprocessing/analysis, not another round of model training. It should be reported as a new analysis version alongside, not silently substituted for, the saved 1.5-second maintenance window.

7.1 Representational analyses

This section assumes knowledge of RSA and defines only the estimators and aggregation used in this repository. RDM code is in `brainalign_wm/analysis/rdm.py`, and comparison and reliability code is in `brainalign_wm/analysis/rsa.py`.

7.1.1 Distance estimators used

There is no single distance measure used everywhere.

Analysis	Model distance	Neural distance
Main maintenance RSA	load-stratified crossnobis	load-stratified crossnobis
Main probe RSA	crossnobis	cross-validated squared Euclidean
Encoder baseline	load-stratified crossnobis	load-stratified crossnobis
Binary maintenance baseline	binary same/different label distance	ordinary Euclidean
Maintenance task-control analysis	ordinary Euclidean	ordinary Euclidean
Probe task-control analysis	binary or Hamming label distance	cross-validated squared Euclidean

RDM upper triangles are compared with Spearman correlation. Crossnobis is therefore not applied consistently across every analysis.

For a cross-validated distance between conditions a and b , the code estimates their mean difference in separate folds and takes the cross-product. With two folds,

$$d_{ab} = (\mu_a^{(1)} - \mu_b^{(1)})^\top \Sigma^{-1} (\mu_a^{(2)} - \mu_b^{(2)}).$$

The maintenance model uses up to four folds. The maintenance brain estimate uses two folds. Cross-load entries are set to missing and excluded from the RSA correlation.

The cross-product is useful because independent noise tends not to agree across folds. Crossnobis can therefore be negative when the underlying separation is near zero; it is an unbiased estimate, not a distance forced to be nonnegative. `crossnobis_rdm` and `stratified_crossnobis_rdm` in `brainalign_wm/analysis/rdm.py` construct the RDMs, and `compare_rdms` in `brainalign_wm/analysis/rsa.py` compares their upper triangles.

7.1.2 Main maintenance RSA Scientific question. Does the model distinguish remembered contents in the same relative geometry as the recorded population during the delay?

Input rows. One epoch-averaged model vector and one epoch-averaged neural vector for every trial in one session.

Condition label. The primary label is

$$c_i = (L_i, \text{sort}(\text{remembered image IDs}_i)).$$

If exact image sets do not repeat often enough, the code uses

$$c_i = (L_i, \text{sort}(\text{remembered categories}_i)).$$

Conditions represented by fewer than two trials are removed. At least four shared conditions and at least three finite within-load RDM pairs are required.

RDMs. For each session s ,

$$D_{\text{model},s} \in \mathbb{R}^{C_s \times C_s}, \quad D_{\text{brain},s} \in \mathbb{R}^{C_s \times C_s}.$$

Both use crossnobis distances within load. Cross-load cells are omitted.

Aggregation. Each session yields

$$r_s = \rho_{\text{Spearman}}(\text{vec}_{\Delta} D_{\text{model},s}, \text{vec}_{\Delta} D_{\text{brain},s}).$$

The run result is the mean signed correlation across usable sessions:

$$\bar{r}_{\text{maint}} = \frac{1}{S_{\text{usable}}} \sum_s r_s.$$

For the headline pooled analysis, $S_{\text{usable}} = 65$.

Within one session, Spearman correlation uses every finite upper-triangle pair from the within-load blocks. Loads with more surviving condition pairs contribute more entries. The code does not compute three load-specific RSA values and then average them. No per-load or per-condition-pair result is saved.

Reliability normalization. Split-half neural RDM reliability is estimated within each session. Session reliabilities are averaged, and the run-level signed RSA is divided by that average ceiling. The normalized value is clipped to $[0, 1]$. Individual session correlations are not clipped before averaging.

Status. Executed for all 192 main runs, 65 sessions, eight region selections, and worker or manager subpopulations where applicable. The long table has 125,824 data rows.

Result. Across supervised cells, signed means range from -0.0630 to $+0.0073$. The mean over supervised run-session observations is -0.019 . Mean neural reliability is 0.643 . The current estimator therefore reports little signed maintenance alignment. Its detection limit has not yet been measured because the sensitivity analysis has no saved result.

7.1.3 Main probe RSA Scientific question. Does the model organize decision-period states like the neural population?

Condition label. Every trial is assigned to one of 12 conditions:

$$c_i = (L_i, I_i, A_i) \in \{1, 2, 3\} \times \{0, 1\} \times \{0, 1\},$$

where I_i says whether the probe is in the remembered set and A_i says whether the patient answered correctly.

Model RDM. A trained model has the same units in every replayed trial, so its 8,820 trial vectors can be combined. Conditions require at least eight trials. Up to four folds give one

$$D_{\text{model}} \in \mathbb{R}^{12 \times 12}$$

crossnobis RDM per run.

Neural RDM. Neurons from different sessions were not recorded simultaneously. For each neuron, condition means are computed only from trials in that neuron’s own session. These estimates are then concatenated across neurons and sessions into a pseudopopulation with shape

$$\text{fold} \times \text{condition} \times \text{neuron} = 4 \times 12 \times p.$$

Missing session-condition entries remain missing. Because a joint cross-session noise covariance is not observable, the brain RDM uses cross-validated squared Euclidean distance rather than crossnobis.

Aggregation. Each run and region yields one model RDM, one neural pseudopopulation RDM, and one RSA correlation. The output is not an average of 65 per-session RSA values, and it is not a 65×65 RDM.

With 12 complete conditions, the correlation uses the 66 upper-triangle condition pairs together. The code does not emit a separate RSA for each load, membership value, correctness value, individual condition, session, or patient.

Status. Executed for all 192 runs and eight region selections. Every run has all 12 shared conditions.

Result by ROI and signal. Maintenance values are means over run-session rows for the all-unit representation. Probe values are means over one pseudopopulation result per run. “All” combines 120 SUP and 72 RL runs and is therefore weighted toward SUP.

ROI	Maintenance, all	Maintenance, SUP	Maintenance, RL	Probe, all	Probe, SUP	Probe, RL
Pooled	-0.0026	-0.0192	+0.0251	0.4696	0.5030	0.4139
MFC	-0.0261	-0.0395	-0.0038	0.3969	0.4119	0.3719
MTL	-0.0014	-0.0116	+0.0155	0.1634	0.1885	0.1216
dACC	+0.0095	+0.0084	+0.0113	0.3101	0.3202	0.2932
preSMA	-0.0326	-0.0418	-0.0173	0.4150	0.4238	0.4004
vmPFC	-0.0172	-0.0238	-0.0062	0.0893	0.1140	0.0481
Amygdala	+0.0056	+0.0008	+0.0135	0.0819	0.0889	0.0703
Hippocampus	-0.0134	-0.0208	-0.0011	0.0282	0.0518	-0.0112

The corresponding saved normalized row means are:

ROI	Maintenance, all	Maintenance, SUP	Maintenance, RL	Probe, all	Probe, SUP	Probe, RL
Pooled	0.1275	0.1104	0.1561	0.9234	0.9780	0.8325
MFC	0.1088	0.0957	0.1306	0.7546	0.7731	0.7237
MTL	0.1294	0.1179	0.1487	0.4574	0.5173	0.3576
dACC	0.1318	0.1283	0.1377	0.7908	0.8081	0.7618
preSMA	0.1187	0.1150	0.1248	0.7138	0.7216	0.7007
vmPFC	0.1141	0.1005	0.1368	0.2024	0.2379	0.1433
Amygdala	0.1515	0.1478	0.1577	0.1932	0.2048	0.1739
Hippocampus	0.1274	0.1174	0.1441	0.1161	0.1526	0.0553

For maintenance, this table averages normalized session rows after each row has been clipped. It is not the same estimator as dividing a signed run mean by a mean ceiling. For probe, each row is already one run-level pseudopopulation result.

The pooled probe neural ceiling is 0.506. SUP pooled-region cell means range from 0.4566 to 0.5182. RL values are lower and more variable, especially for the two-seed hierarchical cells.

The same variables define both the 12 conditions and obvious task structure. The saved score has not yet been shown to exceed load, membership, correctness, and their combined structure.

7.1.4 Cell and training-signal RSA means These are seed means from results/alignment_results.csv. Maintenance is the equal-weight mean of session RSA values within each run. Probe is one pseudopopulation RSA value per run. No per-load RSA exists for either epoch.

Runs included	Maintenance raw	Maintenance normalized	Probe raw	Probe normalized
All 192	-0.00259	0.03667	0.46958	0.92344
120 SUP	-0.01918	0.00671	0.50301	0.97801
72 RL	+0.02507	0.08661	0.41386	0.83249

These are run-weighted means. The all-run row does not give SUP and RL equal weight.

Signal	Cell	Seeds	Maintenance raw	Maintenance normalized	Probe raw	Probe normalized
SUP	M00000	8	-0.0331	0.0000	+0.5064	0.9830
SUP	M00001	8	-0.0266	0.0024	+0.5100	0.9941
SUP	M00010	8	-0.0173	0.0056	+0.5172	0.9905
SUP	M00011	8	-0.0140	0.0081	+0.5081	0.9872
SUP	M00100	8	-0.0630	0.0000	+0.4685	0.9259
SUP	M01000	8	-0.0298	0.0056	+0.4888	0.9525
SUP	M01111	8	-0.0479	0.0000	+0.4566	0.8971
SUP	M10000	8	-0.0243	0.0038	+0.5167	0.9960
SUP	M10001	8	-0.0153	0.0049	+0.5160	0.9992
SUP	M10010	8	+0.0073	0.0169	+0.5122	0.9913
SUP	M10111	8	-0.0102	0.0103	+0.5151	0.9959
SUP	M11011	8	+0.0007	0.0161	+0.5182	0.9964
SUP	M11101	8	-0.0050	0.0069	+0.4953	0.9766
SUP	M11110	8	-0.0012	0.0168	+0.5104	0.9975
SUP	M11111	8	-0.0082	0.0032	+0.5056	0.9868
RL	M00000	8	-0.0252	0.0017	+0.4895	0.9575
RL	M00001	8	-0.0178	0.0071	+0.4871	0.9516
RL	M00010	8	-0.0419	0.0056	+0.4630	0.9151
RL	M00011	8	-0.0172	0.0458	+0.4510	0.8915
RL	M00100	8	-0.0426	0.0037	+0.5151	0.9888
RL	M01000	8	+0.0291	0.0998	+0.2868	0.6453
RL	M01111	8	-0.0176	0.0412	+0.3836	0.8088
RL	M10000	2	+0.2140	0.3327	+0.3938	0.7784
RL	M10001	2	+0.0902	0.1670	+0.3949	0.7804
RL	M10010	2	+0.1104	0.1876	+0.4043	0.7991
RL	M10111	2	+0.2221	0.3452	+0.3504	0.6927
RL	M11011	2	+0.2285	0.3552	+0.2074	0.4322
RL	M11101	2	+0.1030	0.1850	+0.4752	0.9393
RL	M11110	2	+0.2289	0.3558	+0.4658	0.8461
RL	M11111	2	+0.2381	0.3702	-0.0977	0.0669

The large positive maintenance values in several hierarchical RL cells occur in models with near-chance behavior. They do not establish useful memory coding and should not be combined with SUP.

7.1.5 Reliability and normalized RSA The reported normalized score is

$$r_{\text{norm}} = \text{clip} \left(\frac{r_{\text{raw}}}{c_{\text{upper}}}, 0, 1 \right).$$

The ceiling estimates reproducibility of the neural RDM. It does not convert RSA into explained neural variance. Clipping maps all negative values to zero and all above-ceiling values to one. Signed raw RSA is therefore the primary quantity for interpretation.

7.1.6 Worker and manager RSA Hierarchical models expose three representations: all recurrent units, worker units, and manager units. Maintenance RSA compares worker and manager representations with MTL and MFC recordings. The intended comparisons are worker–MTL and manager–MFC; the cross-comparisons are retained.

Data unit. Remembered-content conditions within one recording session.

RDM and aggregation. This uses the same per-session condition RDMs and the same crossnobis estimator as main maintenance RSA.

Output. One signed RSA value for each run, session, model subpopulation, and neural region.

Status. Executed. Results are stored one row per session in results/alignment_by_session.csv, and a figure exists. No saved patient-aware inferential summary compares the four mappings.

Result. Values below average run-session rows across the 64 SUP and 16 RL hierarchical runs. “Pooled” worker or manager rows were not generated; subpopulation RSA exists only for named regions. Probe RSA has no worker or manager variant.

Model population	Neural ROI	All runs	SUP	RL
Worker	MFC	-0.0111	-0.0375	+0.0946
Worker	MTL	+0.0125	-0.0087	+0.0971
Worker	dACC	+0.0134	+0.0067	+0.0402
Worker	preSMA	-0.0281	-0.0405	+0.0215
Worker	vmPFC	-0.0183	-0.0245	+0.0068
Worker	Amygdala	+0.0120	+0.0028	+0.0488
Worker	Hippocampus	-0.0112	-0.0201	+0.0247
Manager	MFC	-0.0554	-0.0701	+0.0034
Manager	MTL	-0.0503	-0.0629	-0.0001
Manager	dACC	+0.0058	+0.0022	+0.0202
Manager	preSMA	-0.0272	-0.0333	-0.0029
Manager	vmPFC	-0.0223	-0.0258	-0.0082
Manager	Amygdala	-0.0202	-0.0246	-0.0026
Manager	Hippocampus	-0.0237	-0.0289	-0.0032

These are session-weighted descriptive means. There is no equal-patient mean or saved worker-versus-manager inferential contrast.

7.1.7 Frozen-encoder baseline This baseline averages the frozen ResNet features of the remembered images, labels trials by remembered content, and applies the same per-session maintenance condition estimator.

Question. How much maintenance alignment is already present in the visual features, without recurrent memory processing?

Data unit and output. Remembered-content conditions within one session; one signed and normalized RSA value per session and region.

Status. Executed for one shared control over available sessions and regions.

Result. Across 387 session-region rows, mean raw RSA is 0.0202 and mean normalized RSA is 0.1599. The pooled-ROI rows alone average 0.0252 raw and 0.1216 normalized. This control is shared across models, so it has no SUP, RL, cell, seed, worker, or manager variant.

7.1.8 Binary task-structure baseline This maintenance control uses one row per trial. For trials i and j ,

$$D_{ij}^{\text{task}} = \begin{cases} 0, & \text{same load and same remembered set,} \\ 1, & \text{otherwise.} \end{cases}$$

The brain RDM is

$$D_{ij}^{\text{brain}} = \|x_i - x_j\|_2.$$

The analysis must remain trial-level. If each remembered set were collapsed to one condition, every off-diagonal task distance would be 1 and the task RDM would have no useful variance.

Output. One signed and normalized RSA value per session and region.

Status. Executed for the shared maintenance baseline.

Result. Across 387 session-region rows, mean raw RSA is 0.00761 and mean normalized RSA is 0.0154. Pooled-ROI rows alone average 0.0103 raw and 0.0170 normalized. This result does not control the probe RSA because the probe uses different conditions and a pseudopopulation.

The two maintenance baselines have the following ROI means. Session counts differ because not every session contains each anatomical region.

Baseline	ROI	Sessions	Raw RSA	Normalized RSA
Encoder	Pooled	65	+0.0252	0.1216
Encoder	MFC	45	+0.0103	0.1515
Encoder	MTL	65	+0.0436	0.1691
Encoder	dACC	32	-0.0147	0.1125
Encoder	preSMA	39	-0.0060	0.1563
Encoder	vmPFC	27	+0.0512	0.2494
Encoder	Amygdala	58	+0.0219	0.1698
Encoder	Hippocampus	56	+0.0169	0.1764
Binary task	Pooled	65	+0.0103	0.0170
Binary task	MFC	45	+0.0105	0.0194
Binary task	MTL	65	+0.0077	0.0150
Binary task	dACC	32	+0.0040	0.0127
Binary task	preSMA	39	+0.0105	0.0218
Binary task	vmPFC	27	+0.0023	0.0089
Binary task	Amygdala	58	+0.0081	0.0156
Binary task	Hippocampus	56	+0.0042	0.0110

Neither baseline has an equal-patient summary. The grand mean over 387 rows weights ROIs by their available-session counts.

7.1.9 Untrained-network control An untrained M11110 network is replayed on the same trials.

Data unit and output. The same maintenance session-condition and probe 12-condition analyses as a trained run; control RSA values for one random initialization.

Status. Executed.

Result. Maintenance raw RSA is -0.09685 and normalized RSA is 0. Probe normalized RSA is 0.09098. The trained M11110_SUP_s0 run has maintenance raw RSA 0.01654 and probe raw RSA 0.52218, with normalized probe RSA clipped to 1.

This control addresses random network structure. It does not remove the task variables used to define probe conditions.

7.1.10 Reflection-shuffle intervention For $M = 1$ models, the sequence $R_1, \dots, R_T$ is shuffled within each trial, preserving its values while breaking their alignment to trial events. The effect is

$$\Delta r = r_{\text{normal}} - r_{\text{shuffled}}.$$

Data unit and output. One trained $M = 1$ run; the difference between normal and shuffled maintenance RSA.

Status. Executed for 72 runs.

Result. Signal-level means are:

Signal	Runs	Normal raw RSA	Shuffled raw RSA	Raw effect	Normalized effect
SUP	48	-0.01523	-0.01673	+0.00150	+0.00153
RL	24	+0.07037	+0.07103	-0.00066	+0.00383

Cell-level lesion effects are:

Signal	Cell	Runs	Raw effect	Normalized effect
SUP	M01000	8	+0.00059	+0.00046
SUP	M01111	8	-0.00131	+0.00001
SUP	M11011	8	+0.00314	+0.00272
SUP	M11101	8	+0.00179	+0.00089
SUP	M11110	8	+0.00419	+0.00541
SUP	M11111	8	+0.00057	-0.00031
RL	M01000	8	-0.00674	+0.00730
RL	M01111	8	+0.00306	+0.00100
RL	M11011	2	-0.00165	-0.00257
RL	M11101	2	-0.00561	-0.00641
RL	M11110	2	+0.01592	+0.02474
RL	M11111	2	-0.00188	-0.00292

Only $M = 1$ cells are applicable. The saved intervention is pooled over maintenance sessions and regions; it has no ROI, load, worker, manager, or patient-specific result.

7.1.11 Maintenance label permutation Remembered-content labels can be permuted within load while holding model activity fixed. This preserves load and destroys the mapping between activity and remembered content.

Data unit and output. Per-session maintenance condition RDMs; a null distribution of RSA under label exchange.

Status. Implemented in the alignment code. No standalone campaign result table is saved.

7.1.12 Probe task-structure and semipartial controls The probe control constructs four 12×12 label RDMs:

$$D_{ab}^L = \mathbb{K}[L_a \neq L_b], \quad D_{ab}^I = \mathbb{K}[I_a \neq I_b], \quad D_{ab}^A = \mathbb{K}[A_a \neq A_b],$$

$$D_{ab}^{\text{combined}} = \frac{\mathbb{K}[L_a \neq L_b] + \mathbb{K}[I_a \neq I_b] + \mathbb{K}[A_a \neq A_b]}{3}.$$

Each is compared with the same brain pseudopopulation RDM as the model. A semipartial analysis residualizes the combined task RDM from both model and brain RDM vectors, then correlates the residuals.

The out-of-sample version divides sessions into two groups. One group estimates task-related structure and the other evaluates residual model–brain alignment; the direction is then reversed. Splitting by session keeps all trials from one recording together.

Data unit and output. Twelve probe conditions; task-only RSA values and a held-out residual model–brain RSA value per run and region.

Status. Implemented in uncommitted code and tested. The expected results/task_structure_baselines.csv file is absent. There is no saved result.

7.1.13 Maintenance sensitivity analysis This positive control generates synthetic model patterns containing a known fraction of the real neural geometry plus isotropic noise. It passes them through the actual session-wise maintenance estimator, with the real condition counts and missingness. Recovery and patient-bootstrap intervals are measured across injected effect sizes.

Data unit and output. Real session and condition structure with simulated representations; recovered RSA and patient-bootstrap interval at each injected effect size.

Status. Implemented in uncommitted code and tested. No saved sensitivity curve exists. The smallest detectable maintenance alignment is therefore unknown.

7.1.14 Epoch-by-method analysis Maintenance and probe currently differ in epoch, condition definition, session aggregation, and distance estimator. The control crosses epoch with analysis method:

Neural epoch	Per-session, load-stratified method	Pooled, coarse-condition method
Maintenance	main maintenance analysis	method control
Probe	method control	main probe analysis

Data unit and output. The units required by each estimator; a four-cell table that separates epoch from method.

Status. Implemented in uncommitted code and tested. No saved result exists. The large maintenance–probe difference cannot yet be assigned specifically to task epoch.

7.2 Architectural contrasts and uncertainty

Contrasts pair models by training seed. SUP and RL are analyzed separately. For maintenance RSA, bootstrap samples draw patients and carry all sessions from one patient together. This preserves the repeated-session structure.

Data unit. A matched pair of trained runs at the seed level. Maintenance uncertainty additionally resamples patients as clusters.

Output. A paired mechanism effect, bootstrap interval, two-sided p -value, and FDR decision.

Patients are deterministically assigned to 38 discovery and 19 confirmation patients. Results for both splits already exist, so the confirmation set has been inspected and is no longer prospectively untouched.

At equal training budget, the completed SUP campaign gives:

Outcome	Contrast	Effect	95% patient-bootstrap interval	Primary-family FDR
Maintenance RSA	add P to baseline	-0.0299	[-0.0544, -0.0090]	yes
Maintenance RSA	full minus no S	+0.0397	[+0.0122, +0.0687]	yes
Probe RSA	add P to baseline	-0.0379	[-0.0580, -0.0177]	yes
Probe RSA	full minus no S	+0.0490	[+0.0275, +0.0713]	yes
Probe RSA	full minus no M	-0.00946	[-0.0157, -0.0028]	yes

The isolated $S + T$ cell exceeds baseline maintenance RSA by 0.0404, with interval [0.0155, 0.0693]. This comparison lies outside the saved primary add-one and remove-one FDR family.

The equal-budget RL rows that pass the same primary-family FDR flag are:

Outcome	Contrast	Effect	95% bootstrap interval	Seeds
Maintenance RSA	add S to baseline	+0.2542	[+0.1785, +0.3390]	2
Maintenance RSA	full minus no S	+0.3082	[+0.2384, +0.3847]	2
Probe RSA	add S to baseline	-0.0820	[-0.1279, -0.0361]	2
Probe RSA	full minus no S	-0.6013	[-0.8015, -0.4010]	2

Outcome	Contrast	Effect	95% bootstrap interval	Seeds
Probe RSA	full minus no M	-0.4481	[-0.6293, -0.2669]	2
Probe RSA	full minus no P	-0.3051	[-0.3696, -0.2405]	2
Probe RSA	full minus no T	-0.5729	[-0.7087, -0.4371]	2
Probe RSA	full minus no D	-0.5634	[-0.6133, -0.5135]	2

These RL effects compare two-seed hierarchical models that often perform near chance. They should be read as associations within failed or weak learners, rather than evidence about successful task solutions.

The full model has higher alignment than the no-hierarchy model, while the no-hierarchy model has higher load-3 accuracy: 0.9480 versus 0.9303. This shows why performance and alignment must be analyzed separately.

Across the 15 SUP cell means, the number of active bits has Spearman correlation approximately 0.625 with maintenance RSA, -0.026 with probe RSA, and approximately 0.24 with load-3 accuracy. Counting bits is only descriptive because the mechanisms are not equivalent units and the design is not a full factorial.

For completeness, the following tables average every run with a bit off or on. These are descriptive marginal means from an unbalanced, selected-cell design. They are not the paired causal contrasts in the preceding table.

Signal	Bit	Load-3 off	Load-3 on	On minus off
SUP	S	0.9306	0.9223	-0.0083
SUP	M	0.9261	0.9264	+0.0003
SUP	P	0.9231	0.9309	+0.0079
SUP	T	0.9176	0.9338	+0.0162
SUP	D	0.9239	0.9282	+0.0043
RL	S	0.8625	0.5886	-0.2738
RL	M	0.8453	0.7143	-0.1309
RL	P	0.8084	0.7881	-0.0203
RL	T	0.8186	0.7826	-0.0360
RL	D	0.8172	0.7842	-0.0330

Signal	Bit	Maintenance off	Maintenance on	On minus off
SUP	S	-0.0331	-0.0070	+0.0261
SUP	M	-0.0218	-0.0152	+0.0066
SUP	P	-0.0169	-0.0226	-0.0056
SUP	T	-0.0281	-0.0113	+0.0168
SUP	D	-0.0230	-0.0158	+0.0072
RL	S	-0.0190	+0.1794	+0.1984
RL	M	+0.0024	+0.0704	+0.0680
RL	P	+0.0146	+0.0459	+0.0313
RL	T	+0.0095	+0.0424	+0.0329
RL	D	+0.0121	+0.0395	+0.0274

Signal	Bit	Probe off	Probe on	On minus off
SUP	S	0.4937	0.5112	+0.0175
SUP	M	0.5078	0.4958	-0.0120
SUP	P	0.5104	0.4919	-0.0185
SUP	T	0.5002	0.5054	+0.0052
SUP	D	0.5029	0.5031	+0.0003
RL	S	0.4395	0.3243	-0.1152
RL	M	0.4653	0.3110	-0.1542
RL	P	0.4213	0.3991	-0.0222
RL	T	0.4409	0.3836	-0.0574

Signal	Bit	Probe off	Probe on	On minus off
RL	<i>D</i>	0.4359	0.3893	-0.0466

The RL on/off means are strongly affected by the near-chance hierarchical cells and their smaller seed count. No ROI-specific, worker-specific, or manager-specific knob contrasts are saved. No mechanism contrast is saved separately by load because RSA itself is not stored per load.

Status and available variants. results/alignment_contrasts.csv has 316 rows. It contains SUP and RL separately; maintenance and probe dependent variables; add-one, remove-one, two-arm, interaction, and active-bit families; equal-budget and load-3-within-0.05 matching; and all-patient estimates. Maintenance additionally has discovery and confirmation folds because it retains session and patient observations. Probe has only the all-patient pseudopopulation result. At-criterion alignment files exist, but no at-criterion contrast rows are saved. No contrast is broken down by load, ROI, worker, or manager.

7.3 Temporal coding and persistent activity

7.3.1 Cross-temporal decoding Decoding receives labeled activity rows (X, y) . A classifier fit at time t predicts the same labels from held-out activity at t' . The diagonal asks whether the label is decodable at a time; off-diagonal entries ask whether the same readout generalizes across time. High off-diagonal accuracy supports a stable code only when diagonal accuracy itself exceeds a label-permutation null. `cross_temporal_decoding` and `cross_temporal_stability` in `brainalign_wm/analysis/cross_temporal.py` implement this distinction.

A classifier trained at time t and tested at time t' yields $A_{tt'}$. The old summary was

$$S_{\text{old}} = \frac{\text{mean}_{t \neq t'} A_{tt'}}{\text{mean}_t A_{tt}}.$$

This ratio can approach one when both diagonal and off-diagonal accuracy are at chance. It therefore does not by itself establish a stable informative code.

The revised implementation reports diagonal accuracy, empirical chance, 100 label permutations, a permutation p -value, and a stability ratio only when diagonal decoding exceeds the null.

Data unit. Trials with one activity vector at each time point, analyzed separately for model and neural data.

Output. A train-time by test-time accuracy matrix, absolute diagonal and off-diagonal accuracy, permutation statistics, and a gated stability summary.

Saved old result. The CSV covers all 192 runs and 20 sessions per run. Mean old stability is 0.7058 for SUP, 0.8564 for RL, and 0.9898 for neural activity.

Status. The saved file predates the corrected implementation and cannot support an interpretation of neural stability. Regeneration is pending.

The old output retains every cell and seed but stores only run-level model and neural means. It has no load, ROI, worker, manager, individual-session, or patient-level result. The neural mean is identical in the SUP and RL summaries because the same neural data are reused.

7.3.2 Persistent activity The old unit-level index was

$$P_u^{\text{old}} = \frac{\mu_{u,\text{preferred}} - \mu_{u,\text{fixation}}}{|\mu_{u,\text{fixation}}| + 1}.$$

The constant 1 has different meaning for firing rates and bounded GRU states. The model-brain comparison is therefore scale-dependent.

The corrected estimator selects the preferred load using one half of the trials, evaluates the difference on the other half, and divides by the unit's trial-to-trial standard deviation:

$$P_u^{\text{std}} = \frac{\mu_{u,\text{preferred,test}} - \mu_{u,\text{fixation,test}}}{s_u}.$$

Data unit. One recurrent unit or neuron, with fixation and maintenance activity split across trials.

Output. One standardized cross-validated persistence effect per unit, followed by population summaries.

Saved old result. Means are 0.0509 for SUP, 0.0394 for RL, and 0.1356 for neural units.

Status. The corrected function exists and is tested. No corrected result file exists.

The old output retains every cell and seed but only the final model and neural means per run. It has no load, ROI, worker, manager, session, or patient breakdown. The neural mean is repeated across training signals.

7.3.3 Memory–sensation cross-decoding The function trains decoders for remembered items and tests them on currently viewed probe representations, and vice versa. It asks whether sensory and memory codes occupy shared or different subspaces.

Data unit and output. Trial-by-time activity with item labels; cross-epoch decoding accuracy in both directions.

Status. Implemented and tested as a function. The standard campaign runner does not call it, and no result file exists.

7.4 Encoding models and variance decomposition

7.4.1 Model-to-neuron and neuron-to-model encoding RSA compares distance geometry after discarding coordinate identity. Encoding keeps a predictive mapping: given trial matrix X and target activity Y , ridge regression fits $\hat{Y} = XW + b$ while penalizing large coefficients. Its penalty is selected inside training folds and its $R^2 = 1 - SS_{\text{residual}}/SS_{\text{total}}$ is scored on held-out trials. Negative held-out R^2 means the mapping predicted worse than the target mean, not an invalid percentage. `encoding_r2` in `brainalign_wm/analysis/encoding.py` is the exact implementation.

Within each session, cross-validated ridge regression predicts neural activity from model activity and also predicts model activity from neural activity.

Data unit. Matched trial vectors within one recording session.

Output. Cross-validated R^2 in each direction, with a reported reliability normalization for model-to-neuron prediction.

Status. Executed for every main run and all 65 sessions at the pooled-region level: 12,480 rows.

Result.

Signal	Run-session rows	Model to neuron R^2	Normalized model to	
			neuron	Neuron to model R^2
All	12,480	-0.000518	0.000090	-0.129455
SUP	7,800	-0.000600	0.000105	-0.126870
RL	4,680	-0.000383	0.000067	-0.133763

The normalization reuses a session RSA reliability proxy rather than a reliability estimate specific to each neuron’s regression target. Raw cross-validated R^2 is the clearer result.

Rows retain cell, seed, session, and patient identifiers, so cell and patient summaries are derivable. They are not saved as separate summary or contrast tables. Only the pooled ROI and all-unit model representation were analyzed. There is no per-load, named-ROI, worker, or manager result.

7.4.2 Condition-marginalized principal-component analysis This saved analysis partitions condition-mean activity into item, load, time, and interaction effects with ANOVA-style marginalization, then applies ordinary PCA within each marginalization. A large load fraction therefore means the population varies with load; it does not show that model and neural load axes are aligned. `marginalize` and `dpca_components` in `brainalign_wm/analysis/dpca.py` implement this descriptive decomposition.

It is not a full dPCA fit: the implementation does not include the regularized encoder/decoder estimator described by Kobak et al. (2016).

Data unit. Condition-mean model or neural activity indexed by item, load, and time.

Output. Fractions of total variance assigned to each marginalization. The ordinary PCA axes are not validated full-dPCA axes, and the analysis does not align model and neural axes.

Status. Executed for every main run and all 65 sessions: 12,480 rows.

Result.

Signal	Model item	Model load	Neural item	Neural load
All	0.0771	0.1018	0.01846	0.03990
SUP	0.08215	0.09862	0.01846	0.03990
RL	0.06865	0.10714	0.01846	0.03990

The neural means are identical across SUP and RL because the same neural session values are repeated beside each model run. Rows retain cell, seed, session, and patient identifiers, but no separate cell, equal-patient, ROI, worker, manager, or per-load summary is saved. These are descriptive row means; no formal model–brain contrast is saved.

7.5 State-space geometry

7.5.1 Trajectory speed and participation ratio A trial is a path through hidden-state space. Speed measures successive state displacement; participation ratio is effective dimensionality. For eigenvalues $(4, 1, 0, \dots)$, $PR = (4 + 1)^2 / (4^2 + 1^2) = 25/17$, so variance occupies more than one direction but not two equally used ones. `mean_speed` and `pca_participation_ratio` in `brainalign_wm/analysis/geometry.py` compute the saved measures.

Maintenance trajectory speed is

$$v = \text{mean}_{i,t} \|h_{i,t+1} - h_{i,t}\|_2.$$

Participation ratio is computed from PCA eigenvalues:

$$PR = \frac{(\sum_k \lambda_k)^2}{\sum_k \lambda_k^2}.$$

Data unit. A trained run’s single-trial maintenance trajectories.

Output. One mean speed and one PCA participation ratio per run.

Status. Both measures were executed for all 192 main runs. Four legacy rows contain topology-loading errors.

Result.

Signal	Runs	Mean speed	Mean participation ratio
All main runs	192	0.8960	8.3535
SUP	120	1.1616	8.9341
RL	72	0.4534	7.3839

Across main runs, speed ranges from 0.069 to 2.793 and participation ratio from 1.154 to 20.255. Rows retain cell and seed, so cell and bit summaries can be derived, but no inferential mechanism contrast is saved. The output has no load, ROI, session, patient, worker, or manager split.

7.5.2 Other implemented geometry functions The geometry module also contains:

- rotation of content and context axes;
- cross-temporal generalization area under the curve;
- stable-versus-switching unit classification;
- dynamic mode decomposition;
- dominant dynamical directions;
- content–context orthogonalization;
- task-irrelevant decoding;

- correct-versus-error geometry;
- serial-position decoding;
- causal state perturbations;
- an on-demand linear-quadratic regulator.

These functions are implemented and tested. The standard runner does not reconstruct all required labels or interventions, so no campaign result tables exist.

Data unit and output. Depending on the function, trial trajectories, labeled states, or perturbed states; rotation, decoding, dynamical-system, or causal-effect summaries.

Because no standard results exist, there are also no SUP/RL, cell, load, subpopulation, session, patient, or ROI summaries for these functions.

7.6 Network properties

7.6.1 Weight entropy The main output discretizes recurrent weights into 50 histogram bins and computes Shannon entropy:

$$H(W) = - \sum_b p_b \log_2 p_b.$$

Data unit and output. One recurrent population in one trained run; one entropy value.

Status. Executed in the network-properties output.

Result. Mean entropy is 3.6136 bits for flat cores over 112 runs, 1.9211 for workers over 80 hierarchical runs, and 4.4846 for managers over 80 runs.

7.6.2 Effective connectivity and modularity For a GRU, the absolute recurrent matrices from the reset, update, and candidate blocks are summed. The diagonal is removed and the graph is symmetrized. Weighted Louvain modularity is then computed.

Data unit and output. One effective recurrent graph per population and run; one modularity value.

Status. Executed in the network-properties output.

Result. Mean modularity is 0.0500 for flat cores, 0.0903 for workers, and 0.0288 for managers.

7.6.3 Small-worldness The graph is reduced to its strongest 10% of edges, subsampled to at most 64 nodes, and restricted to its largest connected component. Its clustering and path length are compared with rewired random graphs:

$$\sigma = \frac{C/C_{\text{rand}}}{L/L_{\text{rand}}}.$$

Data unit and output. One thresholded recurrent graph per population and run; one small-worldness ratio.

Status. Executed in the network-properties output.

Result. Mean σ is 1.2348 for flat cores, 1.5762 for workers, and 1.1637 for managers.

The estimator uses two random reference graphs and two rewiring iterations. These values are coarse and should not be treated as precise graph statistics. The definition follows Watts and Strogatz (1998).

7.6.4 Degree assortativity Degree assortativity is computed on the thresholded graph.

Data unit and output. One thresholded recurrent graph per compatible run; one degree-assortativity coefficient.

Status and result. The geometry output contains 112 values, with mean -0.1439 and range -0.4687 to 0.0784 . It is not included in the main network-properties JSONL.

7.6.5 Mixed selectivity For each unit, condition-related variance is decomposed into load, item, and load-by-item interaction sums of squares. The unit index is

$$MS_u = \frac{SS_{u,interaction}}{SS_{u,load} + SS_{u,item} + SS_{u,interaction}}.$$

Data unit and output. One unit’s condition means, followed by a population mean per run.

Status and result. Executed for all 192 runs. Mean population mixed selectivity is 0.3359, with range 0.0707–0.5931. No saved mechanism contrast exists.

The complete saved population-by-signal summary is:

Signal	Population	Runs	Entropy	Modularity	Small-worldness	Run-level mixed selectivity
SUP	Flat	56	3.4472	0.0478	1.1359	0.3263
SUP	Worker	64	1.6915	0.0978	1.5755	0.3263
SUP	Manager	64	4.5974	0.0305	1.3027	0.3263
RL	Flat	56	3.7801	0.0522	1.3337	0.3518
RL	Worker	16	2.8394	0.0606	1.5790	0.3518
RL	Manager	16	4.0334	0.0221	0.6077	0.3518

Mixed selectivity is one all-recurrent-population value per run, so it repeats in the population rows above only to show its signal mean. It was not computed separately for worker and manager. Network rows retain cell and seed, but there are no saved cell-level contrasts. Load, ROI, session, and patient are not applicable to weight metrics and are not saved for mixed selectivity.

The implementations are in `brainalign_wm/analysis/network_properties.py`.

7.7 Fixed points and local dynamics

A fixed point is a state that maps to itself under one specified transition; it is not an observed trial-average state. The Jacobian says how a tiny perturbation changes on the next step: eigenvalue magnitudes below one shrink it and values above one expand it. Input, gate bias, and plastic trace must be held fixed before asking about $F(h)$, because the task-driven system is otherwise not a single time-invariant map. `find_fixed_points` and `summarize_fixed_points` in `brainalign_wm/analysis/attractors.py` implement the search and summary.

For a recurrent transition F , candidate fixed points minimize

$$q(h) = \frac{1}{2} \|F(h) - h\|_2^2.$$

The search starts from observed maintenance states and jittered versions of them. Converged candidates are deduplicated. At each point, the code computes

$$J = \frac{\partial F}{\partial h}.$$

A point is stable when

$$\max_k |\lambda_k(J)| < 1.$$

Values within 10^{-3} of unit modulus are labeled marginal. A dominant complex eigenvalue marks oscillatory local dynamics. For hierarchical models, worker and manager blocks of the joint Jacobian are also summarized.

External input, gate bias, and fast plastic state are held at representative fixed values during the local transition analysis. A returned fixed point is therefore conditional on those held values, rather than a fixed point of the complete time-varying task.

Data unit. One trained recurrent transition function, seeded from observed and jittered maintenance states.

Output. Deduplicated fixed points, residual speed, stability class, and Jacobian eigenvalues.

Status. Executed for 196 runs, including four legacy controls.

Result. The mean detected count is 31.32, with range 0–192. Mean stable count is 0.0816, mean marginal count is 31.21, and mean oscillatory count is 0.005. A maximum eigenvalue modulus exists for 101 runs; its mean is 0.9834 and range is 0.7288–1.0259. Most detected points are marginal.

Separating the 192 main runs from the four legacy controls gives:

Signal	Runs	Fixed	Stable	Marginal	Oscillatory	Maximum modulus
SUP	120	6.9500	0.0000	6.9500	0.0000	1.0002
RL	72	70.3472	0.2222	70.0556	0.0139	0.9734

The maximum-modulus means use only runs with a finite value. Hierarchical JSON rows also store worker- and manager-block stable counts and maximum moduli. Across the 16 hierarchical RL runs, mean worker and manager stable counts are each 0.8125; their mean maximum moduli are 0.7875 and 0.8896. Across the 64 hierarchical SUP runs, both stable-count means are zero and the block maximum-modulus fields are missing. No formal SUP–RL or mechanism contrast is saved.

Rows retain cell and seed. There is no load, ROI, session, or patient dimension because this is a model-only run-level analysis.

The implementation follows the fixed-point approach of Sussillo and Barak (2013) and is in `brainalign_wm/analysis/attractors.py`.

7.8 Relations among outcome measures

The cross-dependent-variable table combines accuracy, maintenance and probe RSA, modularity, small-worldness, mixed selectivity, persistence, and fixed-point measures. Code computes Pearson and Spearman correlations with bootstrap intervals and a regression of RSA on accuracy with a seed random intercept.

Data unit. One trained run, with run-level outcomes joined by run identifier.

Output. Pairwise correlations, bootstrap intervals, and accuracy-adjusted RSA regression coefficients.

Status. The 192-row input table exists. Correlation output is printed but not preserved as a dedicated result file. The persistence column still uses the old scale-dependent estimator.

Result. The verified descriptive cell-level relations are the active-bit correlations reported in Section 7.2. A complete corrected result awaits regenerated persistence values.

7.9 Additional experiments and coded substrates

7.9.1 Capacity curve Flat baseline models were trained with

$$H \in \{2, 4, 8, 16, 32, 64, 128, 256\},$$

using one seed per size.

Data unit and output. One trained run at each hidden size; load-wise accuracy and participation ratio.

Status and result. Executed with one seed per size. No size passed the full behavioral criterion. Load-3 accuracy ranges from 0.51 to 0.68. Participation ratio increases from 1.0 at $H = 2$ to 9.23 at $H = 256$, while behavior does not improve monotonically.

7.9.2 Teacher–student distillation A high-performing teacher was distilled into students with 1–64 recurrent units. Representation match is RSA against the teacher over six task conditions.

Data unit and output. One student size; behavioral accuracy and a six-condition student–teacher RSA.

Status and result. Executed. Students with 1–4 units remain near chance and have teacher RSA from 0 to 0.0143. A 16-unit student reaches 0.9714 but fails the behavioral criterion. The 32- and 64-unit students pass, with RSA 0.9857 and 0.9786.

7.9.3 Yang-task reference model A pretrained 256-unit leaky RNN was evaluated on three Yang-task variants, 64 trials each.

Data unit and output. One task-specific set of 64 model trials; one participation ratio per task.

Status and result. Executed. Participation ratio is 6.79 for dm1, 9.38 for contextdm1, and 4.97 for multidm. This is an external geometry reference, not a neural-alignment result.

7.9.4 Identity-catch memory-demand experiment The proposed experiment compares M00000 and M10010 under ordinary recognition and recognition with occasional delayed category reports about the first encoded item.

Data unit and planned output. One architecture, training condition, and seed; recognition accuracy, identity-report accuracy, and maintenance RSA.

Status. The task, runner, and two 100-step smoke runs exist. Their accuracy is near chance, as expected after only 100 steps. The planned 16 full runs have not been launched.

7.9.5 Alternative recurrent substrates The repository implements three flat leaky-rate networks:

Substrate	Mechanism	Main configured size
Excitatory/inhibitory rate RNN	hard outgoing signs for 80% excitatory and 20% inhibitory units	241
Dynamic-synapse RNN	per-trial facilitating and depressing synaptic states	241
Low-rank rate RNN	rank-16 recurrent matrix	241

The leaky update uses a two-step time constant. The dynamic-synapse model divides units equally between facilitation-dominant and depression-dominant kinetics. Parameter matching and gradient-flow diagnostics are implemented.

Data unit and planned output. One trained substrate and seed; behavior, geometry, and the same neural-alignment measures as the GRU campaign.

Status. Code and tests exist. No full training campaign or neural-alignment result exists for these substrates.

7.9.6 Other supplementary mechanisms The code also contains an ungated tanh RNN, a three-gate PBWM-style manager, optional recurrent noise, an activity-energy penalty, flat-core dropout, an L_1 weight penalty, a readout-scale sweep, and a positive lognormal recurrent-weight initialization. These are supplementary controls rather than cells in the completed 15-cell campaign. Some have smoke or diagnostic coverage; no complete common neural-alignment table covers all of them.

Data unit and output. One supplementary configuration and seed; behavior and, where run, the standard post-training measures.

7.9.7 Proposed causal tests Two project-specific tests remain proposals:

1. independently perturb hidden activity and the Hebbian trace during maintenance to locate the stored information;
2. test whether maintenance alignment predicts robustness to longer delays, distractors, and unseen item combinations among behaviorally matched networks.

Data unit and planned output. One trained run under a controlled perturbation or challenge condition; a causal performance or alignment change.

No campaign driver or result exists for either test.

8. Backlog and references

8.1 Implementation map and reproducibility

Topic	Code or result
Configuration	configs/config.yaml
Trial generation	brainalign_wm/tasks/sternberg.py
Curriculum	brainalign_wm/tasks/curriculum.py
GRU and plastic GRU	brainalign_wm/models/gru_cell.py
Worker-manager core	brainalign_wm/models/hrl.py
Reflective gate	brainalign_wm/mechanisms/reflective_gate.py
Training and losses	brainalign_wm/training/train.py
Recorded-trial replay	brainalign_wm/training/generate_activity_logs.py
Neural adapter and binning	brainalign_wm/neural/adapters/dandi_nwb.py
RDM estimators	brainalign_wm/analysis/rdm.py
RSA and ceilings	brainalign_wm/analysis/rsa.py
Main analysis driver	brainalign_wm/analysis/run_all.py
Probe pseudopopulation	brainalign_wm/analysis/pseudopopulation.py
Task controls and sensitivity	brainalign_wm/analysis/task_structure.py;
	brainalign_wm/analysis/measurement_validation.py
Temporal decoding	brainalign_wm/analysis/cross_temporal.py
Persistence	brainalign_wm/analysis/persistence.py
Geometry	brainalign_wm/analysis/geometry.py
Network properties	brainalign_wm/analysis/network_properties.py
Fixed points	brainalign_wm/analysis/attractors.py
Statistical contrasts	brainalign_wm/analysis/contrasts.py
Main alignment results	results/alignment_results.csv
Per-session maintenance results	results/alignment_by_session.csv
Regional probe results	results/alignment_probe_by_region.csv
Contrast results	results/alignment_contrasts.csv
Old dynamics results	results/dynamics_persistence.csv
Encoding and condition-marginalized PCA	results/encoding_results.csv; results/dpca_results.csv
Geometry and topology	results/geometry_results.csv;
	results/network_properties.jsonl
Fixed points	results/attractor_properties.jsonl

The saved project record reports 430 passing tests and no failures at the time of that run. Numerical warnings arose in synthetic mixed-effects fits, and Gymnasium emitted environment-metadata warnings. This statement is historical evidence from the snapshot, not a claim that this documentation revision reran the suite.

The working tree is not clean. Measurement-validation code, persistence code, and the main runner have uncommitted changes. Several result files predate those changes. “Implemented now” and “executed to produce the saved table” are therefore separate statuses throughout this document.

8.2 Results backlog derived from the project note

The note *There is a credible paper.txt* is folded into this section as concrete analysis requirements.

Required result	Current evidence	Remaining work
Show whether probe RSA exceeds load, membership, correctness, and combined task structure	main probe RSA exists	run and save probe task controls and held-out semipartial RSA
Establish the detection limit of maintenance RSA	signed session-wise RSA exists	run and save the injected-signal sensitivity curve
Separate epoch effects from estimator effects	2×2 implementation exists	generate its result table
Repair temporal stability	corrected decoder and permutation test exist	regenerate results; do not interpret old ratio
Repair persistence comparison	cross-validated standardized estimator exists	regenerate results; retire old cross-scale values
Extend neural delay coverage	existing epoch-wise GRU RSA snapshot is retained	replay/analyze the full recorded delay with frozen checkpoints; no model retraining is required
Timestamp quality control	timestamp audit found one nonpositive 000469 maintenance delay	quarantine or repair the malformed trial explicitly, retain per-source timing metadata, and rerun only timing-sensitive analyses if its inclusion changes them
Audit reward and correctness semantics	training derives probe correctness from the sampled model action; replay latches the recorded trial's correct field for feedback modulation	verify adapter-label provenance and document the two scalar meanings before treating reflective replay as a causal mechanism test
Estimate architectural effects at proper replication levels	seed pairing and patient bootstrap exist	add the missing equal-performance or at-criterion contrast table
Keep SUP and RL separate	implemented	preserve separation; hierarchical RL has only two seeds
Use independent confirmation	current confirmation patients have been examined	freeze the analysis and use unused patients or another dataset
Test a mechanism directly	reflection shuffle is complete but weak	run identity-demand or hidden-state versus synaptic-trace perturbation profile and cost the targeted next studies before committing compute; do not infer that every architecture or timing change requires a 196-run replacement grid
Audit the next training budget (A7)	structural-synapse matching and saved 192-run snapshot exist	select a budgeted, behaviorally viable representative subset before any common neural analysis; no blanket full-grid retraining is implied
Compare mechanistic recurrent substrates	three rate substrates exist	audit the pipeline, then use representative paired checkpoint continuations with a WM-only control; match total additional updates and report the mixed branch's lower WM exposure
Test broad cognitive competence (A9)	six-task diet code exists; the chosen 10-entry continuation schema is not yet implemented	

Recommended execution order:

1. generate probe task controls, maintenance sensitivity, and epoch-by-method tables;
2. regenerate temporal decoding and persistence with the corrected estimators;
3. compute equal-performance checkpoint contrasts;
4. freeze the primary analysis before a new confirmation dataset;
5. run one targeted memory-demand or memory-location experiment.

8.3 References

- Cho et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. ACL Anthology.
- Kriegeskorte, Mur, and Bandettini (2008). Representational similarity analysis. *Frontiers in Systems Neuroscience*.
- Walther et al. (2016). Reliability of dissimilarity measures for multi-voxel pattern analysis. *NeuroImage*.
- Miconi, Clune, and Stanley (2018). Differentiable plasticity. PMLR.
- Bellec et al. (2020). A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature Communications*.
- Kobak et al. (2016). Demixed principal component analysis of neural population data. *eLife*.
- Sussillo and Barak (2013). Opening the black box: low-dimensional dynamics in high-dimensional recurrent neural networks. *Neural Computation*.
- Watts and Strogatz (1998). Collective dynamics of small-world networks. *Nature*.
- Khona and Chandra (2023). Winning the lottery with continuous sparsification for language modeling. *arXiv*.
- Daume et al. (2024). Control of working memory by phase-amplitude coupling of human hippocampal neurons. *Nature*.
- Kyzar et al. (2024). Human single-neuron activity dataset during a visual working-memory task. *Data descriptor*.